

Improving Graph Neural Network Expressivity via Subgraph Isomorphism Counting

Giorgos Bouritsas¹, Fabrizio Frasca,
Stefanos Zafeiriou¹, *Member, IEEE*, and Michael M. Bronstein, *Fellow, IEEE*

Abstract—While Graph Neural Networks (GNNs) have achieved remarkable results in a variety of applications, recent studies exposed important shortcomings in their ability to capture the structure of the underlying graph. It has been shown that the expressive power of standard GNNs is bounded by the Weisfeiler-Leman (WL) graph isomorphism test, from which they inherit proven limitations such as the inability to detect and count graph substructures. On the other hand, there is significant empirical evidence, e.g. in network science and bioinformatics, that substructures are often intimately related to downstream tasks. To this end, we propose “Graph Substructure Networks” (GSN), a topologically-aware message passing scheme based on substructure encoding. We theoretically analyse the expressive power of our architecture, showing that it is strictly more expressive than the WL test, and provide sufficient conditions for universality. Importantly, we do not attempt to adhere to the WL hierarchy; this allows us to retain multiple attractive properties of standard GNNs such as locality and linear network complexity, while being able to disambiguate even hard instances of graph isomorphism. We perform an extensive experimental evaluation on graph classification and regression tasks and obtain state-of-the-art results in diverse real-world settings including molecular graphs and social networks.

Index Terms—Graph classification, graph isomorphism, graph neural networks, graph substructures, network analysis, neural network expressivity

1 INTRODUCTION

THE field of graph representation learning has undergone a rapid growth in the past few years. In particular, Graph Neural Networks (GNNs), a family of neural architectures designed for irregularly structured data, have been successfully applied to problems ranging from social networks and recommender systems [1] to bioinformatics [2], [3], chemistry [4], [5], [6] and physics [7], [8], to name a few. Most GNN architectures are based on message passing [5], where the representation of each vertex is iteratively updated by aggregating information from its neighbours.

- Giorgos Bouritsas and Stefanos Zafeiriou are with the Department of Computing, Imperial College London, SW7 2AZ London, U.K. E-mail: {g.bouritsas, s.zafeiriou}@imperial.ac.uk.
- Fabrizio Frasca is with the Department of Computing, Imperial College London, SW7 2AZ, U.K., and also with the Twitter Cortex, W1B 5DL London, U.K. E-mail: ffrasca@twitter.com.
- Michael M. Bronstein is with the Department of Computer Science, University of Oxford, OX1 3QD, U.K., with the IDSIA, University of Lugano, 6900 Lugano, Switzerland, and also with the Twitter Cortex, W1B 5DL London, U.K. E-mail: m.bronstein@imperial.ac.uk.

Manuscript received 16 June 2021; revised 18 Dec. 2021; accepted 13 Feb. 2022. Date of publication 24 Feb. 2022; date of current version 5 Dec. 2022.

The work of Giorgos Bouritsas and Michael M. Bronstein were supported by ERC Consolidator under Grant 724228 - LEMAN. The work of Giorgos Bouritsas was supported by PhD scholarship from the Department of Computing, Imperial College London. The work of Stefanos Zafeiriou was supported in part by the EPSRC Fellowship DEFORM: Large Scale Shape Analysis of Deformable Models of Humans under Grant EP/S010203/1 and in part by Google Faculty Award. The work of Michael M. Bronstein was supported by the Google Faculty Award and the Royal Society Wolfson Research Merit Award.

(Corresponding author: Giorgos Bouritsas.)

Recommended for acceptance by J. M. Solomon.

Digital Object Identifier no. 10.1109/TPAMI.2022.3154319

A crucial difference from traditional neural networks operating on grid-structured data is the absence of canonical ordering of the vertices in a graph. To address this, the aggregation function is constructed to be invariant to neighbourhood permutations and, as a consequence, to graph isomorphism. This kind of symmetry is not always desirable and thus different inductive biases that disambiguate the neighbours have been proposed. For instance, in geometric graphs, such as 3D molecular graphs and meshes, directional biases are usually employed in order to model the positional information of the vertices [9], [10], [11], [12], [13]; for proteins, ordering information is used to disambiguate amino-acids at different positions in the sequence [14]; in multi-relational knowledge graphs, a different aggregation is performed for each relation type [15].

The structure of the graph itself does not usually explicitly take part in the aggregation function. In fact, most models rely on multiple message passing steps as a means for each node to discover the global structure of the graph. However, since message-passing GNNs are at most as powerful as the Weisfeiler Leman test (WL) [16], [17], they are limited in their abilities to adequately exploit the graph structure, e.g. by counting substructures [18], [19]. This uncovers a crucial limitation of GNNs, as substructures have been widely recognised as important in the study of complex networks. For example, in molecular chemistry, functional groups and rings are related to a plethora of chemical properties, while cliques are related to protein complexes in Protein-Protein Interaction networks and community structure in social networks, respectively [20], [21].

Therefore, three major questions arise when designing GNN architectures: (a) How to go beyond *isotropic*, i.e., locally symmetric, aggregation functions? (b) How to ensure

that GNNs are aware of the *structural characteristics* of the graph? (c) How to achieve the above two without sacrificing *invariance to isomorphism* and hence the ability of GNNs to generalise?

In this work we attempt to simultaneously provide an answer to the above. We propose to break local symmetries by introducing structural information in the aggregation function, hence addressing (a) and (b). In particular, the contribution of each neighbour (*message*) is transformed differently depending on its structural relationship with the central node. This relationship is expressed by counting the appearance of certain substructures. Since substructure counts are *vertex invariants*, i.e. they are invariant to vertex permutations, it is easy to see that the resulting GNN will be invariant to isomorphism, hence also addressing (c). Moreover, by choosing the substructures, one can provide the model with different inductive biases, based on the graph distribution at hand.

We characterise the expressivity of our message-passing scheme, coined as *Graph Substructure Network (GSN)*, showing that GSN is strictly more expressive than traditional GNNs for the vast majority of substructures, while retaining the locality of message passing, as opposed to higher-order methods [17], [22], [23], [24] that follow the WL hierarchy (see Section 2). In the limit, our model can yield a unique representation for every isomorphism class and is thus universal. We provide an extensive experimental evaluation on hard instances of graph isomorphism testing (strongly regular graphs), as well as on real-world networks from the social and biological domains, including the recently introduced large-scale benchmarks [25], [26]. We observe that when choosing the structural inductive biases based on domain-specific knowledge, GSN achieves state-of-the-art results.

2 PRELIMINARIES

Let $G = (\mathcal{V}_G, \mathcal{E}_G)$ be a graph with vertex set $\mathcal{V}_G$ and edge set $\mathcal{E}_G$, directed or undirected, with n vertices and m edges. A subgraph $G_S = (\mathcal{V}_{G_S}, \mathcal{E}_{G_S})$ of G is any graph with $\mathcal{V}_{G_S} \subseteq \mathcal{V}_G$, $\mathcal{E}_{G_S} \subseteq \mathcal{E}_G$. When $\mathcal{E}_{G_S}$ includes all the edges of G with endpoints in $\mathcal{V}_{G_S}$, i.e., $\mathcal{E}_{G_S} = \mathcal{E}_G \cap (\mathcal{V}_{G_S} \times \mathcal{V}_{G_S})$, the subgraph is said to be *induced*.

2.1 Isomorphism & Automorphism

Two graphs G, H are *isomorphic* (denoted $H \simeq G$), if there exists an adjacency-preserving bijective mapping (*isomorphism*) $f: \mathcal{V}_G \rightarrow \mathcal{V}_H$, i.e., $(v, u) \in \mathcal{E}_G$ iff $(f(v), f(u)) \in \mathcal{E}_H$. Given some small graph H , the *subgraph isomorphism* problem amounts to finding a subgraph G_S of G such that $G_S \simeq H$. An *automorphism* of H is an isomorphism that maps H onto itself. The set of all automorphisms forms a group, the *automorphism group* of the graph, denoted as $\text{Aut}(H)$, which contains all possible graph symmetries.

The automorphism group yields a partition of the vertices into disjoint subsets of $\mathcal{V}_H$ called *orbits*. Intuitively, this concept allows us to group the vertices based on their *structural roles*, e.g. the endpoint vertices of a path, or all the vertices of a cycle (see Fig. 1). Formally, the orbit of a vertex $v \in \mathcal{V}_H$ is the set of vertices to which it can be mapped via an automorphism: $\text{Orb}(v) = \{u \in \mathcal{V}_H \mid \exists g \in \text{Aut}(H) \text{ s.t. } g(u) = v\}$, and the set of all orbits $\mathcal{V}_H / \text{Aut}(H) = \{\text{Orb}(v) \mid v \in \mathcal{V}_H\}$ is called

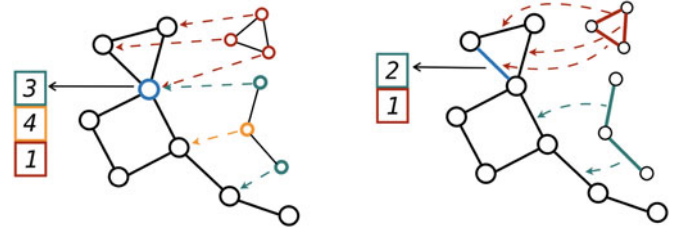


Fig. 1. *Vertex* (left) and *edge* (right) induced subgraph counting for a 3-cycle and a 3-path. Counts are reported for the blue vertex on the left and for the blue edge on the right. Different colors depict orbits.

the *quotient* of the action of the automorphism group on $\mathcal{V}_H$. We denote the elements of this set as $\{O_{H,1}^V, \dots, O_{H,d_H}^V\}$, where d_H is the cardinality of the quotient.

Analogously, we define edge structural roles via *edge automorphisms*, i.e., bijective mappings from the edge set onto itself, that preserve edge adjacency (two edges are adjacent if they share a common endpoint). In particular, every vertex automorphism g induces an edge automorphism by mapping each edge (u, v) to $(g(u), g(v))$.¹ In the same way as before, we construct the edge automorphism group, from which we deduce the partition of the edge set in *edge orbits* $\{O_{H,1}^E, O_{H,2}^E, \dots, O_{H,d_H}^E\}$.

2.2 Weisfeiler-Leman Tests

The *Weisfeiler-Leman graph-isomorphism test* [28], also known as naive vertex refinement, *1-WL*, or just *WL*, is a fast heuristic to decide if two graphs are isomorphic or not. The WL test proceeds as follows: every vertex v is initially assigned a colour c_v^0 that is later iteratively refined by aggregating neighbouring information:

$$c_v^{t+1} = \text{HASH}(c_v^t, c_u^t u \in \mathcal{N}(v)), \quad (1)$$

where $\cdot$ denotes a multiset (a set that allows element repetitions) and $\mathcal{N}(v)$ is the neighbourhood of v . The WL algorithm terminates when the colours stop changing, and outputs a histogram of colours. Two graphs with different histograms are non-isomorphic; if the histograms are identical, the graphs are possibly, but not necessarily, isomorphic. Note that the neighbour aggregation in the WL test is a form of message passing, and GNNs are the learnable analogue.

A series of works on improving GNN expressivity mimic the higher-order generalisations of WL, known as k -WL and k -Folklore WL (WL hierarchy) and operate on k -tuples of vertices (see Appendix B.1). The $(k+1)$ -FWL is strictly stronger than k -FWL, k -FWL is as strong as $(k+1)$ -WL and 2-FWL is strictly stronger than the simple 1-WL test.

3 GRAPH SUBSTRUCTURE NETWORKS

Graphs consist of vertices (or edges) with repeated structural roles. Thus, it is natural for a neural network to treat them in a similar manner, akin to weight sharing between local patches in CNNs for images [29] or positional encodings in language models for sequential data [30], [31], [32].

1. Note that the edge automorphism group is larger than that of induced automorphisms, but strictly larger only for 3 trivial cases [27]. However, induced automorphisms provide a more natural way to express edge structural roles.

Nevertheless, GNNs are usually unaware of the vertices' different structural roles, since all vertices are treated equally when performing local operations. Despite the initial intuition that the neural network would be able to discover these roles by constructing deeper architectures, it has been shown that GNNs are ill-suited for this purpose and are blind to the existence of structural properties, e.g. triangles or larger cycles [18], [19].

To this end, we propose to explicitly encode structural roles as part of message passing, in order to capture richer topological properties. Our method draws inspiration from [33], where it was shown that GNNs become universal when the vertices in the graph are uniquely identified, i.e. when they are equipped with different features. However, it is not clear how to choose these identifiers in a permutation equivariant way. Structural roles, when treated as identifiers, although not necessarily unique, are permutation equivariant, and also more amenable to generalisation due to their repetition across different graphs. Thus, they can constitute a trade-off between uniqueness and generalisation.

3.1 Structural Features

Structural roles are encoded into features by counting the appearance of certain substructures. Let $\mathcal{H} = \{H_1, H_2 \dots H_K\}$ be a set of small (connected) graphs, for example cycles of fixed length or cliques. For each graph $H \in \mathcal{H}$, we first find its isomorphic subgraphs in G denoted as G_S . For each vertex $v \in \mathcal{V}_{G_S}$ we infer its role w.r.t. H by obtaining the orbit of its mapping $f(v)$ in H , $\text{Orb}_H(f(v))$. By counting all the possible appearances of different orbits in v , we obtain the *vertex structural feature* $\mathbf{x}_H^V(v)$ of v , defined as follows. For all $i \in \{1, \dots, d_H\}$:

$$x_{H,i}^V(v) = \left| \left\{ G_S \simeq H \mid v \in \mathcal{V}_{G_S}, f(v) \in O_{H,i}^V \right\} \right|. \quad (2)$$

Note that there exist $|\text{Aut}(H)|$ different functions f that can map a subgraph G_S to H , but any of those can be used to determine the orbit mapping of each vertex v . By combining the counts from different substructures in $\mathcal{H}$ and different orbits, we obtain the feature vector $\mathbf{x}_v^V = [\mathbf{x}_{H_1}^V(v), \dots, \mathbf{x}_{H_K}^V(v)] \in \mathbb{N}^{D \times 1}$ of dimension $D = \sum_{H_i \in \mathcal{H}} d_{H_i}$.

Similarly, we can define *edge structural features* $\mathbf{x}_{H,i}^E(u, v)$ by counting occurrences of edge automorphism orbits:

$$x_{H,i}^E(u, v) = \left| \left\{ G_S \simeq H \mid (u, v) \in \mathcal{E}_{G_S}, (f(u), f(v)) \in O_{H,i}^E \right\} \right|, \quad (3)$$

and the combined edge features $\mathbf{x}_{u,v}^E = [\mathbf{x}_{H_1}^E(u, v), \dots, \mathbf{x}_{H_K}^E(u, v)]$. An example of vertex and edge structural features is illustrated in Fig. 1.

3.2 Structure-Aware Message Passing

The key building block of our architecture is the graph substructure layer, defined in a general manner as a Message Passing Neural Network (MPNN) [5], where now the messages from the neighbouring vertices also contain the structural information. In particular, each vertex v updates its state $\mathbf{h}_v^t$ by combining its previous state with the aggregated messages:

$$\mathbf{h}_v^{t+1} = \text{UP}^{t+1}(\mathbf{h}_v^t, \mathbf{m}_v^{t+1}) \quad (4)$$

$$\mathbf{m}_v^{t+1} = \begin{cases} M^{t+1} \left((\mathbf{h}_v^t, \mathbf{h}_u^t, \mathbf{x}_v^V, \mathbf{x}_u^V, \mathbf{e}_{u,v})_{u \in \mathcal{N}(v)} \right) & (\text{GSN} - \mathbf{v}) \\ \text{or} \\ M^{t+1} \left((\mathbf{h}_v^t, \mathbf{h}_u^t, \mathbf{x}_{u,v}^E, \mathbf{e}_{u,v})_{u \in \mathcal{N}(v)} \right) & (\text{GSN} - \mathbf{e}), \end{cases} \quad (5)$$

where UP^{t+1} is an arbitrary function approximator (e.g. a MLP), M^{t+1} is the neighborhood aggregation function, i.e. an arbitrary function on multisets (e.g., of the form $\sum_{u \in \mathcal{N}(v)} \text{MLP}(\cdot)$) and $\mathbf{e}_{u,v}$ are the edge features. The two variants, named *GSN-v* and *GSN-e*, correspond to vertex- or edge-counts, respectively, which are analogous to absolute and relative positional encodings in language models [35], [36].

It is important to note here that contrary to identifier-based GNNs [33], [37], [38] that obtain universality at the expense of permutation equivariance (since the identifiers are arbitrarily chosen with the sole requirement of being unique), GSNs retain this property, hence they are by construction invariant to isomorphism. This stems from the fact that the process generating our structural identifiers (i.e. subgraph isomorphism) is permutation equivariant itself (proof provided in the Appendix A.1).

3.3 How Powerful are GSNs?

We now turn to the expressive power of GSNs in comparison to MPNNs and the WL tests, a key tool for the theoretical analysis of the expressivity of graph neural networks so far. Since GSN is a generalisation of MPNNs, it is easy to see that it is at least as powerful. Importantly, GSNs have the capacity to learn functions that traditional MPNNs cannot learn. The following observation derives directly from the analysis of the counting abilities of the 1-WL test [18] and its extension to MPNNs [19] (for proofs, see Appendices A2-A4).

Theorem 3.1. *GSN is strictly more powerful than MPNN and the 1-WL test when one of the following holds:*

- *H is any graph except for star graphs of any size, and structural features are inferred by subgraph matching, i.e. we count all subgraphs $G_S \cong H$ for which it holds that $\mathcal{E}_{G_S} \subseteq \mathcal{E}_G$. Or,*
- *H is any graph except for single edges and single vertices, and structural features are inferred by induced subgraph matching, i.e. we count all subgraphs $G_S \cong H$ for which it holds that $\mathcal{E}_{G_S} = \mathcal{E}_G \cap (\mathcal{V}_{G_S} \times \mathcal{V}_{G_S})$.*

Proof. It is easy to see that GSN model class contains MPNNs, and is thus at least as expressive. We can also show that GSN is at least as expressive as 1-WL by repurposing the proof of Theorem 3 in [16] (see Appendix A.2)

Given the first part of the proposition, in order to show that GSNs are strictly more expressive than the 1-WL test, it suffices to show that GSN can distinguish a pair of graphs that 1-WL deems isomorphic. [18] showed that 1-WL, and consequently MPNNs, can count only *forests of stars*. Thus, if the subgraphs are required to be connected, then they can only be star graphs of any size (note that this contains single vertices and single edges).

In addition, [19] showed that 1-WL, and consequently

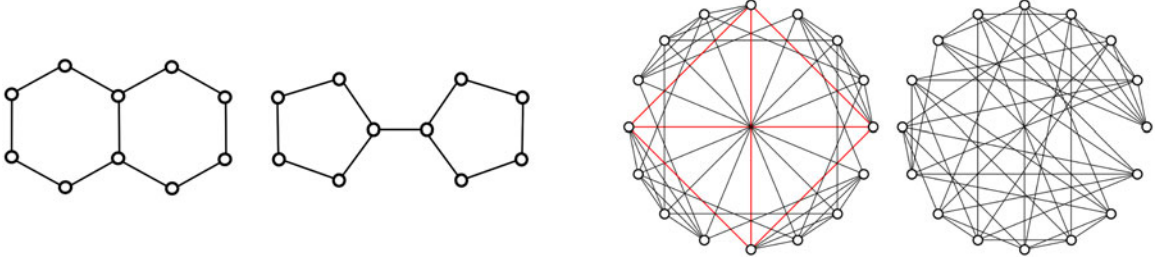


Fig. 2. (Left) *Decalin* and *Bicyclopentyl*: Non-isomorphic molecular graphs than can be distinguished by GSN, but not the by the WL test [34] (vertices represent carbon atoms and edges chemical bonds). (Right) *Rook's 4x4 graph* and the *Shrikhande graph*: the smallest pair of strongly regular non-isomorphic graphs with the same parameters $SR(16,6,2,2)$. GSN can distinguish them with 4-clique counts, while 2-FWL fails.

MPNNs, cannot count any connected **induced** subgraph with 3 or more vertices, i.e. any connected subgraph apart from single vertices and single edges.

If H is a substructure that 1-WL cannot learn to count, i.e. the ones mentioned above, then there is at least one pair of graphs with different number of counts of H , that 1-WL deems isomorphic. Thus, by assigning counting features to the vertices/edges of the two graphs based on appearances of H , a GSN can obtain different representations for G_1 and G_2 by summing up the features. Hence, G_1 , G_2 are deemed non-isomorphic. An example is depicted in Fig. 2 (left), where the two non-isomorphic graphs are distinguishable by GSN via e.g. cycle counting, but not by 1-WL. $\square$

Universality. A natural question that emerges is what are the sufficient conditions under which GSN can solve graph isomorphism. This would entail that GSN is a universal approximator of functions defined on graphs [38], [39]. To address this, we can examine whether there exists a specific substructure collection that can completely characterise each graph. As of today, we are not aware of any results in graph theory that can guarantee the reconstruction of a graph from a smaller collection of its subgraphs. However, the *Reconstruction Conjecture* [40], [41], states that a graph with size $n \geq 3$ can be reconstructed from its vertex-deleted subgraphs (proven for $n \leq 11$ [42]). Consequently, (proof in the Appendix A.3):

Corollary 3.2. *If the Reconstruction Conjecture holds and the substructure collection $\mathcal{H}$ contains all graphs of size $k = n - 1$, then GSN can distinguish all non-isomorphic graphs of size n and is therefore universal.*

GSN-v vs GSN-e. We can also examine the expressive power of the two proposed variants. A crucial observation that we make is that for each graph H in the collection, the vertex structural identifiers can be reconstructed by the corresponding edge identifiers. Thus, we can show that for every GSN-v there exists a GSN-e that can simulate the behaviour of the former (proof in the Appendix A.4).

Theorem 3.3. *For a given subgraph collection $\mathcal{H}$, let C^V the set of functions that can be expressed by a GSN-v with arbitrary depth and with, and C^E the set of functions that can be expressed by a GSN-e with the same properties. Then, it holds that $C^E \supseteq C^V$, or in other words GSN-e is at least as expressive as GSN-v.*

Comparison with higher-order WL tests. Finally, the expressive power of GSN can be compared to higher-

order versions of the WL test. In particular, for each k -th order Folklore WL test in the hierarchy, it is known that there exists a family of graphs that will make the test fail. These are known in the literature as k -isoregular graphs [43], and the most well-known example is the *Strongly Regular* (SR) graph family, for $k = 2$ (more details in Appendix B).

Hence, if we can find a substructure collection that allows GSN to distinguish certain pairs from these families, then this guarantees that the corresponding k -FWL test is no stronger than GSN. In this work, we identify such counterexamples for the 2-FWL test. Formally:

Proposition 3.4. *There exist substructure families with $\mathcal{O}(1)$ size, i.e., independent of the size of the graph n , such that 2-FWL is no stronger than GSN.*

We provide numerous counterexamples that prove this claim. Fig. 2 (right) provides a typical pair of SR graphs that can be distinguished with a 4-clique, while in section 5.1 this is extended to a large-scale study, where other constant size substructures (paths, cycles and cliques) can achieve similar results.

Although it is not clear if there exists a certain substructure collection that results in GSNs that align with the WL hierarchy, we stress that this is not a necessary condition in order to design more powerful GNNs. In particular, despite the increase in expressivity, k -WL tests are not only more computationally involved, but they also process the graph in a non-local fashion. However, locality is presumed to be a strong inductive bias of GNNs and key to their excellent performance in real-world scenarios.

3.4 How to Choose the Substructures?

Expressivity. The Reconstruction Conjecture provides a sufficient, albeit impractical condition for universality. This motivates us to analyse the constant size case $k = \mathcal{O}(1)$ for practical scenarios, similar to the argument put forward for hard instances of graph isomorphism (Proposition 3.4).

In particular, one can count only the most discriminative subgraphs, i.e. the ones that can achieve the maximum possible vertex disambiguation, similarly to identifier-based approaches. Whenever these subgraph counts can provide a unique identification of the vertices, then universality will also hold (Corollary 3.1. in [33]).

We conjecture, that in real-world scenarios the number of subgraphs needed for unique, or near-unique identification, are far fewer than those dictated by Corollary 3.2. This is

TABLE 1
Graph Classification Accuracy on TUD Dataset. **First**, **Second**, **Third** Best Methods are Highlighted. For GSN, the Best Performing Substructure is Shown. * Graph Kernel Methods

Dataset	MUTAG	PTC	Proteins	NCI1	Collab	IMDB-B	IMDB-M
RWK* [96]	79.2±2.1	55.9±0.3	59.6±0.1	>3 days	N/A	N/A	N/A
GK* (k=3) [84]	81.4±1.7	55.7±0.5	71.4±0.31	62.5±0.3	N/A	N/A	N/A
PK* [97]	76.0±2.7	59.5±2.4	73.7±0.7	82.5±0.5	N/A	N/A	N/A
WL kernel* [98]	90.4±5.7	59.9±4.3	75.0±3.1	86.0±1.8	78.9±1.9	73.8±3.9	50.9±3.8
GNTK* [99]	90.0±8.5	67.9±6.9	75.6±4.2	84.2±1.5	83.6±1.0	76.9±3.6	52.8±4.6
DCNN [100]	N/A	N/A	61.3±1.6	56.6±1.0	52.1±0.7	49.1±1.4	33.5±1.4
DGCNN [101]	85.8±1.8	58.6±2.5	75.5±0.9	74.4±0.5	73.8±0.5	70.0±0.9	47.8±0.9
IGN [22]	83.9±13.0	58.5±6.9	76.6±5.5	74.3±2.7	78.3±2.5	72.0±5.5	48.7±3.4
GIN [16]	89.4±5.6	64.6±7.0	76.2±2.8	82.7±1.7	80.2±1.9	75.1±5.1	52.3±2.8
PPGNs [24]	90.6±8.7	66.2±6.6	77.2±4.7	83.2±1.1	81.4±1.4	73.0±5.8	50.5±3.6
Natural GN [72]	89.4±1.60	66.8±1.79	71.7±1.04	82.7±1.35	N/A	74.8±2.01	51.3±1.50
WEGL [102]	N/A	67.5±7.7	76.5±4.2	N/A	80.6±2.0	75.4±5.0	52.3±2.9
GIN+GraphNorm [103]	91.6 ± 6.5	64.9 ± 7.5	77.4 ± 4.9	82.7 ± 1.7	80.2 ± 1.0	76.0 ± 3.7	N/A
GSN-e	90.6±7.5 6 (cycles)	68.2±7.2 6 (cycles)	76.6±5.0 4 (cliques)	83.5± 2.3 15 (cycles)	85.5±1.2 3 (triangles)	77.8±3.3 5 (cliques)	54.3±3.3 5 (cliques)
GSN-v	92.2±7.5 12 (cycles)	67.4±5.7 10 (cycles)	74.59±5.0 4 (cliques)	83.5±2.0 3 (triangles)	82.7±1.5 3 (triangles)	76.8±2.0 4 (cliques)	52.6±3.6 3 (triangles)

consistent with our experimental findings, where we observed that certain small substructures such as paths and trees, significantly improve vertex disambiguation, compared to the initial vertex features (see Fig. 4 (left) and Table 2 in the appendix). As expected this allows for better fitting of the training data, which validates our claim that GNN expressivity improves.

Generalisation. However, none of the above claims can guarantee good generalisation in unseen data. For example, in Fig. 4, we observe that the test set performance does not follow the same trend with train performance when choosing substructures with strong vertex disambiguation. Aiming at better generalisation, it is desirable to make use of substructures for which there is prior knowledge of their importance in certain network distributions and have been observed to be intimately related to various properties. For example, small substructures (graphlets) have been extensively analysed in protein-protein interaction networks [44], triangles and cliques characterise the structure of ego-nets and social networks in general [20], simple cycles (rings) are central in molecular distributions, directed and temporal

motifs have been shown to explain the working mechanisms of gene regulatory networks, biological neural networks, transportation networks and food webs [45], [46], [47].

In Fig. 4 (right), we showcase the importance of these inductive biases: a cycle-based GSN predicting molecular properties achieves smaller generalisation gap compared to a traditional MPNN, while at the same time generalising better with less training data. Choosing the best substructure collection is still an open problem that does not admit a straightforward solution due to its combinatorial nature. Alternatively, various heuristics can be used, e.g., motif frequencies or feature selection strategies. Answering this question is left for future work.

3.5 Complexity

The complexity of GSN comprises two parts: precomputation (substructure counting) and training/testing. The key appealing property is that training and inference are linear w.r.t the number of edges, $\mathcal{O}(|\mathcal{E}|)$, as opposed to higher-order methods [17], [24] with $\mathcal{O}(n^k)$, and [48] with $\mathcal{O}(n^2)$ training complexity and relational pooling [49] with $\mathcal{O}(n!)$ training complexity in absence of approximations.

The worst-case complexity of subgraph enumeration for arbitrary subgraph patterns of size k is $\mathcal{O}(n^k)$, which follows from naively enumerating all vertex sets of size k in the graph. However, this bound can be substantially improved for specific instances of the problem, taking into account the following: (1) The properties of the target graph, e.g., its sparsity. (2) The structure of the subgraph of interest, e.g., cycles and cliques. These substructure families are of particular importance in real-world graphs, social networks and molecules respectively, as will be discussed in the experimental section. (3) The number of occurrences of the patterns in the target graph (*output sensitive* algorithms), e.g., the algorithm in [50] enumerates connected induced subgraphs in $\mathcal{O}(c(G)nm)$ time, where $c(G)$ the number of occurrences of the subgraph of interest. In the Appendix C.1 we provide an overview of these specialised algorithms and their complexities.

In addition, arbitrary subgraph counting/enumeration is widely studied, and many general-purpose algorithms [51],

TABLE 2
MAE in ZINC

Method	MAE	MAE (EF)
GCN [104]	0.469 ± 0.002	-
GIN [16]	0.408 ± 0.008	-
GraphSage [105]	0.410 ± 0.005	-
GAT [106]	0.463 ± 0.002	-
MoNet [10]	0.407 ± 0.007	-
GatedGCN [107]	0.422 ± 0.006	0.363 ± 0.009
MPNN	0.254 ± 0.014	0.209 ± 0.018
MPNN-r	0.322 ± 0.026	0.279 ± 0.023
PNA [108]	0.320 ± 0.032	0.188 ± 0.004
DGN [74]	0.219 ± 0.010	0.168 ± 0.003
GNNML [109]	0.1612 ± 0.006	-
HIMP [110]	-	0.151 ± 0.006
SMP [48]	0.219 ±	0.138 ±
GSN	0.140±0.006	0.115 ± 0.012

[52], [53], [54], [55], [56], [57] provide practical implementations that achieve significant speed-ups using heuristics. As a side note, approximate counting algorithms are also widely used, especially for counting frequent network motifs [58], [59], [60], [61], and can provide a considerable speed-up. Furthermore, recent neural approaches [62], [63] provide fast approximate counting.

In our experiments, we used a general-purpose subgraph isomorphism algorithm. We benchmarked the VF2 algorithm [53], and its recently improved version, the VF3 [54] in our real-world networks and, as expected, in most of the cases, the run-time was considerably smaller than that of the naive enumeration. As discussed above, specialised algorithms are expected to yield further improvements, however this comparison is beyond the scope of our work. In the Appendix C.2, we report the average and total run-times for various datasets and both algorithms (Tables 1 and 2) as well as the empirical scaling w.r.t. n and k (Fig. 1).

4 RELATED WORK

4.1 Expressive Power of GNNs

WL hierarchy. The seminal results in the theoretical analysis of the expressivity of GNNs [16] and k-GNNs [17] established that traditional message passing-based GNNs are at most as powerful as the 1-WL test. [39] showed that graph isomorphism is equivalent to universal invariant function approximation. Higher-order Invariant Graph Networks (IGNs) have been studied in a series of works [22], [23], [24], [39], [64], [65], establishing connections with the WL hierarchy, similarly to [17], [66]. The main drawbacks of these methods are the training and inference time complexity and memory requirements of $\mathcal{O}(n^k)$ and the super-exponential number of parameters (for linear IGNs) making them impractical, as well as their non-local nature making them more prone to overfitting. Finally, [67] also analysed the expressive power of MPNNs and other more powerful variants and provided generalisation bounds.

Unique identifiers. From a different perspective, [68] and [33] showed the connections between GNNs and distributed local algorithms [69], [70], [71] and suggested more powerful alternatives based on either local orderings or unique global identifiers (in the form of random features in [37]) that make GNNs universal. Similarly, [38] propose to use random colorings in order to uniquely identify the vertices. However, these methods lack a principled permutation equivariant way to choose orderings/identifiers. To date this is an open problem in graph theory called *graph canonisation* and it is at least as hard as solving graph isomorphism itself. A possible workaround is proposed in [49], where the authors take into account all possible vertex permutations. However, obviously this quickly becomes intractable ($\mathcal{O}(n!)$) even when considering small-sized graphs.

More expressive permutation equivariant GNNs. Concurrently with our work, other more expressive GNNs have been proposed using equivariant message passing. In [72], the authors propose to linearly transform each message with a different kernel based on the local isomorphism class of the corresponding edge (similar to our definition of structural roles). However, as also noted by the authors, taking into account all possible local isomorphism classes leads to

insufficient weight sharing and hence to overfitting. In contrast, in GSN, usually the substructure collection is small (~ 5 -10 graphs) and the substructures are repetitive in the graph distribution, and as a result generalisation improves. Vignac *et al.* [48] propose a message passing scheme where matrices of order equal to the size of the graph are propagated instead of vectors. This can be perceived as a practical unique identification scheme, but the neural network complexity becomes quadratic in the number of vertices. Finally, [73] and [74] enhance the aggregation function with distance encodings (a strategy more relevant for vertex-level tasks) and graph eigenvectors respectively as alternative symmetry breaking mechanisms. In the experimental section, GSN is compared against these methods in real-world scenarios.

Quantifying expressivity. Solely quantifying the expressive power of GNNs in terms of their ability to distinguish non-isomorphic graphs does not provide the necessary granularity: even the 1-WL test can distinguish almost all (in the probabilistic sense) non-isomorphic graphs [75]. As a result, there have been several efforts to analyse the power of k -WL tests in comparison to other graph invariants [18], [76], [77], [78], while recently [19] approached GNN expressivity by studying their ability to count substructures.

4.2 Substructures in Complex Networks.

The idea of analysing complex networks based on small-scale topological characteristics dates back to the 1970's and the notion of triad census for directed graphs [79]. The seminal paper of [45] coined the term *network motifs* as over-represented subgraph patterns that were shown to characterise certain functional properties of complex networks in systems biology. The closely related concept of *graphlets* [44], [80], [81], [82], different from motifs in being induced subgraphs, has been used to analyse the distribution of real-world networks and as a topological signature for network similarity. Our work is similar in spirit with the *graphlet degree vector* (GDV) [80], a vertex-wise descriptor based on graphlet counting.

Substructures have been also used in the context of ML. In particular, subgraph patterns have been used to define Graph Kernels (GKs) [83], [84], [85], [86], [87], with the most prominent being the graphlet kernel [84]. Motif-based vertex embeddings [88], [89] and diffusion operators [90], [91], [92] that employ adjacency matrices weighted according to motif occurrences, have recently been proposed for graph representation learning. Our formulation provides a unifying framework for these methods and it is the first to analyse their expressive power. Finally, GNNs that operate in larger induced neighbourhoods [93], [94] or higher-order paths [95] have prohibitive complexity since the size of these neighbourhoods typically grows exponentially.

5 EXPERIMENTAL EVALUATION

In the following section we evaluate GSN in comparison to the state-of-the-art in a variety of datasets from different domains. We are interested in practical scenarios where the collection of subgraphs, as well as their size, are kept small. Depending on the dataset domain we experimented with typical substructure families (*cycles*, *paths* and *cliques*) and maximum substructure size k (note that for each setting,

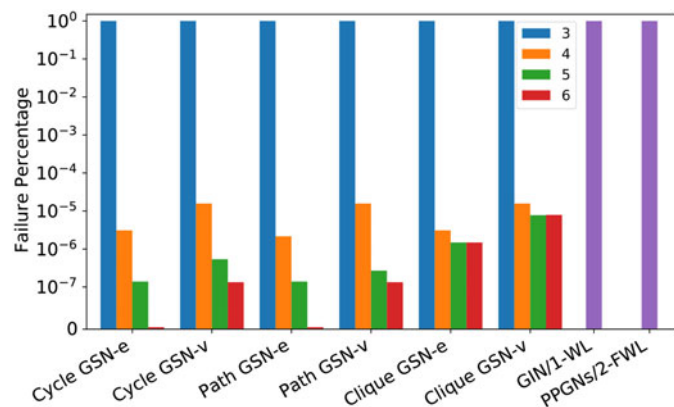


Fig. 3. SR graphs isomorphism test (log scale, smaller values are better). Different colours indicate different substructure sizes.

our substructure collection consists of all the substructures of the family with size $\leq k$. In particular, for networks with community structure (moderate and large clustering coefficient) we used *cliques*, while for molecular graphs we used *cycles*, due to the large frequency of ring structures that are also known to strongly influence molecular properties.

We also experimented with both graphlets and motifs and observed similar performance in most cases. To showcase that structural features can be used as an off-the-shelf strategy to boost GNN performance, we usually choose a base message passing architecture and minimally modify it into a GSN. Unless otherwise stated, the base architecture is a general-purpose MPNN with MLPs used in the message and update functions. Additional implementation details can be found in the Appendix D.

5.1 Synthetic Graph Isomorphism Test

We tested the ability of GSNs to decide if two graphs are non-isomorphic on a collection of Strongly Regular graphs of size up to 35 vertices, attempting to disambiguate pairs with the same number of vertices (for different sizes the problem becomes trivial). As we are only interested in the bias of the architecture itself, we use GSN with random weights to compute graph representations. Two graphs are deemed isomorphic if the Euclidean distance of their representations is smaller than a predefined threshold ϵ . Fig. 3 shows the failure percentage of our isomorphism test when using different graphlet substructures (*cycles*, *paths*, and *cliques*) of varying size k . Interestingly, the number of failure cases of GSN decreases rapidly as we increase k ; cycles and paths of maximum length $k = 6$ are enough to tell apart all the graphs in the dataset. Note that the performance of cliques saturates, possibly because the largest clique in our dataset has 5 vertices. Observe also the discrepancy between GSN-v and GSN-e. In particular, vertex-wise counts do not manage to distinguish all graphs, although missing only a few instances, which is in accordance with Theorem 3.3. Finally, 1-WL [16] and 2-FWL [24] equivalent models demonstrate 100% failure, as expected from theory.

5.2 TUD Graph Classification Benchmarks

We evaluate GSN on datasets from the classical TUD benchmarks. We use seven datasets from the domains of bioinformatics and computational social science and

compare against various GNNs and Graph Kernels. The base architecture that we used is GIN [16]. We follow the same evaluation protocol of [16], performing 10-fold cross-validation and then reporting the performance at the epoch with the best average accuracy across the 10 folds. Table 1 lists all the methods evaluated with the split of [101]. We select our model by tuning architecture and optimisation hyperparameters and substructure related parameters, that is: (i) k , (ii) motifs against graphlets. Following domain evidence we choose the following substructure families: *cycles* for molecules, *cliques* for social networks. Best performing substructures both for GSN-e and GSN-v are reported. As can be seen, our model obtains state-of-the-art performance in most of the datasets, with a considerable margin against the main GNN baselines in some cases.

5.3 ZINC Molecular Graphs

We evaluate GSN on the task of regressing the “penalized water-octanol partition coefficient - $\log P$ ” (see [111], [112], [113] for details) of molecules from the ZINC database [25], [114]. We use structural features obtained with k -cycle counting and report the result of the best performing substructure w.r.t. the validation set.

As dictated by the evaluation protocol of [25], the total number of parameters of the model is approximately 100 K, which is achieved by selecting an appropriate network width (a GSN with 500 K params attains 0.101 ± 0.010 MAE.). The data split is obtained from [25] and the evaluation metric is the Mean Absolute Error (MAE). We compare against a variety of baselines, ranging from traditional message passing NNs to recent more expressive architectures [48], [74], [108], [109] and a molecular-specific one (HIMP) which is based on the junction-tree molecular decomposition [110]. Wherever possible, we compare two variants, one that does not take edge features into account and one that does. In both cases, GSN achieves state-of-the-art results outperforming all the baseline architectures.

5.4 OGB Graph Property Prediction Datasets

We use three graph property prediction datasets from the Open Graph Benchmark (OGB) [26]: ogbg-molhiv and ogbg-molpcba are molecular datasets where the task is graph-level binary classification. In ogbg-molhiv the task is to predict if a molecule inhibits HIV replication or not, while ogbg-molpcba has multiple tasks that we need to optimise for simultaneously. We also evaluate GSN on ogbg-ppa, a dataset of protein-protein association networks, where the task is to predict the taxonomic group of each graph (37-way classification). The underlying distribution of this dataset is significantly different from the other two, since it contains larger and denser graphs with community structure.

In Table 3 we compare against the following baselines: GCN [104] and GIN [16], two conventional GNNs, PNA [108] and DGN [74] two modern GNNs with provably increased expressivity and strong empirical performance, and HIMP, a molecule-specific GNN [110]. There is a plethora of other methods that have reported results in these datasets,

TABLE 3
Test and Validation Performance Metrics for 3 OGB Graph Property Prediction Datasets. **First**, **Second**, **Third**

Method	ogbg-molhiv		ogbg-molpcba		ogbg-ppa	
	Test ROC-AUC	Val. ROC-AUC	Test AP	Val. AP	Test Accuracy	Val. Accuracy
GCN [104]	0.7606 $\pm$ 0.0097	0.8204 $\pm$ 0.0141	0.2020 $\pm$ 0.0024	0.2059 $\pm$ 0.0033	0.6839 $\pm$ 0.0084	0.6497 $\pm$ 0.0034
HIMP [110]	0.7880 $\pm$ 0.0082	N/A	0.2739 $\pm$ 0.0017	N/A	N/A	N/A
PNA [108]	0.7905 $\pm$ 0.0132	0.8519 $\pm$ 0.0099	0.2838 $\pm$ 0.0035	0.2926 $\pm$ 0.0026	N/A	N/A
GIN [16]	0.7558 $\pm$ 0.0140	0.8232 $\pm$ 0.0090	0.2266 $\pm$ 0.0028	0.2305 $\pm$ 0.0027	0.6892 $\pm$ 0.0100	0.6562 $\pm$ 0.0107
GIN+VN [16]	0.7707 $\pm$ 0.0149	0.8479 $\pm$ 0.0068	0.2703 $\pm$ 0.0023	0.2798 $\pm$ 0.0025	0.7037 $\pm$ 0.0107	0.6678 $\pm$ 0.0105
DGN (eigenvectors) [74]	0.7970 $\pm$ 0.0097	0.8470 $\pm$ 0.0047	0.2885 $\pm$ 0.0030	0.2970 $\pm$ 0.0021	N/A	N/A
GSN (GIN base)	0.7606 $\pm$ 0.0174	0.8517 $\pm$ 0.0090	0.2508 $\pm$ 0.0023	0.2542 $\pm$ 0.0021	0.6877 $\pm$ 0.0022	0.6489 $\pm$ 0.0065
GSN (GIN+VN base)	0.7799 $\pm$ 0.0100	0.8658 $\pm$ 0.0084	0.2816 $\pm$ 0.0047	0.2913 $\pm$ 0.0017	0.7119 $\pm$ 0.0120	0.6716 $\pm$ 0.0078
GSN (DGN base)	0.8039 $\pm$ 0.0090	0.8473 $\pm$ 0.0096	N/A	N/A	N/A	N/A

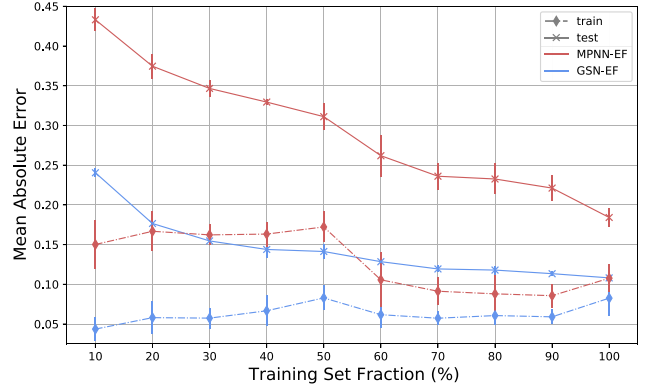
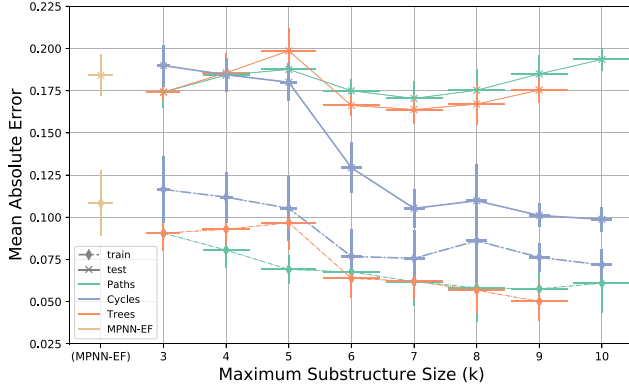


Fig. 4. (Left) Train (dashed) and test (solid) MAEs for Path-, Tree- and Cycle-GSN-EF as a function of the maximum substructure size k . Vertical bars indicate standard deviation; horizontal bars depict disambiguation scores δ . (Right) Train (dashed) and test (solid) MAEs for GSN-EF (blue) and MPNN-EF (red) as a function of the dataset fraction used for training.

sometimes outperforming GSN (see the OGB public leaderboard²). Note that many of them are either domain-specific, or use various tricks which are orthogonal to our work. Hence, an exhaustive comparison is beyond the scope of this experimental section, which is to highlight that GSN can be used as a plug-and-play method and boost the performance of a base architecture without any hyperparameter tuning (except for the maximum substructure size k), rather than to achieve state-of-the-art results. Following this rationale, we choose a base architecture and modify it into a GSN variant by introducing structural features in the aggregation function: cycle counts for the molecular datasets and triangle counts for ogbg-ppa. We use the following base architectures: (a) *GIN* and (b) *GIN-VN*, a variation of GIN that allows for edge features and is extended with a *virtual node*, i.e. a node connected to every node in the graph. For ogbg-molhiv we also used (c) DGN [74] as base, a GNN that propagates messages in an anisotropic manner, based on a predefined graph vector field. Observe that the vector field is an alternative way to break local symmetries. The authors of DGN use vector fields defined by the eigenvectors of the graph, while in our case the vector field is defined by graph substructures. More information can be found in the supplementary material.

Using the evaluators provided by the authors, we report the relevant metric at the epoch with the best validation performance (substructure size is also chosen based on the validation set). By examining the results in Table 3 the following observations can be made, (a) GSN seamlessly improves the

performance of the base architecture, both in the test set and the validation set, sometimes significantly, or, in only one case, is on par. (b) Cyclical substructures are a good inductive bias when learning on molecules, confirming our results on the ZINC dataset, while the same holds for triangles in PPA networks. The latter agrees with our intuition that tasks defined on graphs with community structure correlate with the presence of triangles (or cliques), as was the case for social networks in the TU Datasets experiments. (c) General purpose GNNs benefit from symmetry breaking mechanisms, either in the form of eigenvectors (DGN) or in the form of substructures.

5.5 Ablation Studies

5.5.1 Comparison Between Substructure Collections

In Fig. 4 (left), we compare the training and test error for different substructure families (cycles, paths and non-isomorphic trees – for each experiment we use all the substructures of size $\leq k$ in the family). Additionally, we measure the “uniqueness” of the identifiers each substructure yields as follows: for each graph G in the dataset, we measure the number of unique vertex features u_G (input vertex features concatenated with vertex structural identifiers for GSN-v). Then, we sum them up over the entire training set and divide by the total number of vertices, yielding the disambiguation score $\delta = \frac{\sum_G u_G}{\sum_G |V_G|}$. The disambiguation scores for the different types of substructures are illustrated as horizontal bars in Fig. 4 (the exact values can be found in Appendix D.4, Table 4).

2. https://ogb.stanford.edu/docs/leader_graphprop/

TABLE 4
Comparison Between DeepSets and GSN
With the Same Structural Features

Dataset	DeepSets	# params	GSN	# params
MUTAG	93.3±6.9	3 K	92.8±7.0	3 K
PTC	66.4±6.7	2 K	68.2±7.2	3 K
Proteins	77.8±4.2	3 K	77.8±5.6	3 K
NCII	80.3±2.4	10 K	83.5±2.0	10 K
Collab	80.9±1.6	30 K	85.5±1.2	52 K
IMDB-B	77.1±3.7	51 K	77.8±3.3	65 K
IMDB-M	53.3±3.2	68 K	54.3±3.3	66 K
ZINC	0.288 ±0.003	366 K	0.108 ±0.018	385 K
ogbg-molhiv	77.34±1.46	3.4 M	77.99±1.00	3.3 M

A first thing to notice is that the training error is tightly related to the disambiguation score. As identifiers become more discriminative, the model gains expressive power. On the other hand, the test error is not guaranteed to decrease when the identifiers become more discriminative. For example, although cycles have smaller disambiguation scores, they manage to generalise much better than the other substructures, the performance of which is similar to the baseline architecture (MPNN with MLPs). This is also observed when comparing against [37] (MPNN- r method in Table 2), where, akin to unique identifiers, random features are used to strengthen the expressivity of GNN architectures. This approach also fails to improve the baseline architecture in terms of the performance in the test set. This validates our intuition that unique identifiers can be hard to generalise when chosen in a non-permutation equivariant way and motivates once more the importance of choosing the identifiers not only based on their discriminative power, but also in a way that allows incorporating the appropriate inductive biases. Finally, we observe a substantial jump in performance when using GSN with cycles of size $k \geq 6$. This is not surprising, as cyclical patterns of such sizes (e.g. aromatic rings) are very common in organic molecules.

5.5.2 Generalisation

We repeat the experimental evaluation on ZINC using different fractions of the training set and compare the vanilla MPNN model against GSN. In Fig. 4 (right), we plot the training and test errors of both methods. Regarding the training error, GSN consistently performs better, following our theoretical analysis on its expressive power. More importantly, GSN manages to generalise much better even with a small fraction of the training dataset. Observe that GSN requires only 20% of the samples to achieve approximately the same test error that MPNN achieves when trained on the entire training set.

5.6 Structural Features & Message Passing

We perform an ablation study on the abilities of the structural features to predict the task at hand, when given as input to a graph-agnostic network. In particular, we compare our best performing GSN with a DeepSets model [115] that treats the input features and the structural identifiers as a set. For fairness of evaluation the same hyperparameter search is performed for both models (see

Appendix D.6). Interestingly, as we show in Table 4, our baseline attains particularly strong performance across a variety of datasets and often outperforms other traditional message passing baselines. This demonstrates the benefits of these additional features and motivates their introduction in GNNs, which are unable to compute them. As expected, we observe that applying message passing on top of these features, brings performance improvements in the vast majority of the cases, sometimes considerably, as in the ZINC dataset.

6 CONCLUSION

In this paper, we propose a novel way to design structure-aware graph neural networks. Motivated by the limitations of traditional GNNs to capture important topological properties of the graph, we formulate a message passing scheme enhanced with structural features that are extracted by subgraph isomorphism. We show both theoretically and empirically that our construction leads to improved expressive power and attains state-of-the-art performance in real-world scenarios. In future work, we will further explore the expressivity of GSNs as an alternative to the k -WL tests, as well as their generalisation capabilities. Another important direction is to infer prominent substructures directly from the data and explore the ability of graph neural networks to compose substructures.

REFERENCES

- [1] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 974–983.
- [2] A. Fout, J. Byrd, B. Shariat, and A. Ben-Hur, "Protein interface prediction using graph convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6530–6539.
- [3] P. Gainza *et al.*, "Deciphering interaction fingerprints from protein molecular surfaces using geometric deep learning," *Nature Methods*, vol. 17, no. 2, pp. 184–192, 2020.
- [4] D. Duvenaud *et al.*, "Convolutional networks on graphs for learning molecular fingerprints," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2224–2232.
- [5] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn.*, 2017, vol. 70, pp. 1263–1272.
- [6] B. Sanchez-Lengeling, J. N. Wei, B. K. Lee, R. C. Gerkin, A. Aspuru-Guzik, and A. B. Wiltschko, "The chemistry of smell: Learning generalizable perceptual representations of small molecules," in *Proc. Adv. Neural Inf. Process. Syst. Workshops*, 2019.
- [7] T. N. Kipf, E. Fetaya, K. Wang, M. Welling, and R. S. Zemel, "Neural relational inference for interacting systems," in *Proc. Int. Conf. Mach. Learn.*, *Proc. Mach. Learn. Res.*, 2018, vol. 80, pp. 2693–2702.
- [8] P. Battaglia *et al.*, "Interaction networks for learning about objects, relations and physics," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4502–4510.
- [9] J. Masci, D. Boscaini, M. M. Bronstein, and P. Vandergheynst, "Geodesic convolutional neural networks on riemannian manifolds," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, 2015, pp. 832–840.
- [10] F. Monti, D. Boscaini, J. Masci, E. Rodolà, J. Svoboda, and M. M. Bronstein, "Geometric deep learning on graphs and manifolds using mixture model CNNs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5425–5434.
- [11] G. Bouritsas, S. Bokhnyak, S. Ploumpis, M. Bronstein, and S. Zafeiriou, "Neural 3D morphable models: Spiral convolutional networks for 3D shape representation learning and generation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 7213–7222.
- [12] J. Klicpera, J. Groß, and S. Günnemann, "Directional message passing for molecular graphs," in *Proc. Int. Conf. Learn. Representations*, 2020.

- [13] P. de Haan, M. Weiler, T. Cohen, and M. Welling, "Gauge equivariant mesh CNNs: Anisotropic convolutions on geometric graphs," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [14] J. Ingraham, V. Garg, R. Barzilay, and T. Jaakkola, "Generative models for graph-based protein design," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15 794–15 805.
- [15] M. Schlichtkrull *et al.*, "Modeling relational data with graph convolutional networks," in *Proc. Eur. Semantic Web Conf.*, 2018, vol. 10843, pp. 593–607.
- [16] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [17] C. Morris *et al.*, "Weisfeiler and leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.
- [18] V. Arvind, F. Fuhlbrück, J. Köbler, and O. Verbitsky, "On weisfeiler-leman invariance: Subgraph counts and related graph properties," in *Proc. Fundam. Comput. Theory*, 2019, vol. 11651, pp. 111–125.
- [19] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 10383–10395.
- [20] M. Granovetter, "The strength of weak ties: A network theory revisited," *Proc. Sociol. Theory*, vol. 1, pp. 201–233, 1982.
- [21] M. Girvan and M. E. Newman, "Community structure in social and biological networks," *Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, 2002.
- [22] H. Maron, H. Ben-Hamu, N. Shmair, and Y. Lipman, "Invariant and equivariant graph networks," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [23] H. Maron, E. Fetaya, N. Segol, and Y. Lipman, "On the universality of invariant networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, vol. 97, pp. 4363–4371.
- [24] H. Maron, H. Ben-Hamu, H. Serviansky, and Y. Lipman, "Provably powerful graph networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 2153–2164.
- [25] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," 2020, *arXiv:2003.00982*.
- [26] W. Hu *et al.*, "Open graph benchmark: Datasets for machine learning on graphs," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 22118–22133, 2020.
- [27] H. Whitney, "Congruent graphs and the connectivity of graphs," *Amer. J. Math.*, vol. 54, no. 1, pp. 150–168, 1932.
- [28] B. Weisfeiler and A. Leman, "The reduction of a graph to canonical form and the algebra which appears therein," *NTI, Ser. 2*, vol. 9, pp. 12–16, 1968, [Online]. Available: https://www.iti.zcu.cz/wl2018/pdf/wl_paper_translation.pdf
- [29] Y. LeCun *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural Comput.*, vol. 1, no. 4, pp. 541–551, 1989.
- [30] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2440–2448.
- [31] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, vol. 70, pp. 1243–1252.
- [32] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [33] A. Loukas, "What graph neural networks cannot learn: Depth vs width," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [34] R. Sato, "A survey on the expressive power of graph neural networks," 2020, *arXiv:2003.04078*.
- [35] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2018, pp. 464–468.
- [36] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-XL: Attentive language models beyond a fixed-length context," in *Proc. 57th Conf. Assoc. Comput. Linguistics*, 2019, pp. 2978–2988.
- [37] R. Sato, M. Yamada, and H. Kashima, "Random features strengthen graph neural networks," in *Proc. SIAM Int. Conf. Data Mining*, 2021, pp. 333–341.
- [38] G. Dasoulas, L. D. Santos, K. Scaman, and A. Virmaux, "Coloring graph neural networks for node disambiguation," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 2126–2132.
- [39] Z. Chen, S. Villar, L. Chen, and J. Bruna, "On the equivalence between graph isomorphism testing and function approximation with GNNs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15 868–15 876.
- [40] P. J. Kelly *et al.*, "A congruence theorem for trees," *Pacific J. Math.*, vol. 7, no. 1, pp. 961–968, 1957.
- [41] S. M. Ulam, *A Collection of Mathematical Problems*, vol. 8. New York, NY, USA: Interscience Publishers, 1960.
- [42] B. D. McKay, "Small graphs are reconstructible," *Australas. J. Combinatorics*, vol. 15, pp. 123–126, 1997.
- [43] B. L. Douglas, "The weisfeiler-lehman method and graph isomorphism testing," 2011, *arXiv:1101.5211*.
- [44] N. Pržulj, D. G. Corneil, and I. Jurisica, "Modeling interactome: Scale-free or geometric?," *Bioinformatics*, vol. 20, no. 18, pp. 3508–3515, 2004.
- [45] R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon, "Network motifs: Simple building blocks of complex networks," *Science*, vol. 298, no. 5594, pp. 824–827, 2002.
- [46] A. Paranjape, A. R. Benson, and J. Leskovec, "Motifs in temporal networks," in *Proc. 10th ACM Int. Conf. Web Search Data Mining*, 2017, pp. 601–610.
- [47] A. R. Benson, D. F. Gleich, and J. Leskovec, "Higher-order organization of complex networks," *Science*, vol. 353, no. 6295, pp. 163–166, 2016.
- [48] C. Vignac, A. Loukas, and P. Frossard, "Building powerful and equivariant graph neural networks with structural message-passing," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 14143–14155.
- [49] R. L. Murphy, B. Srinivasan, V. A. Rao, and B. Ribeiro, "Relational pooling for graph representations," in *Proc. Int. Conf. Mach. Learn.*, 2019, vol. 97, pp. 4663–4673.
- [50] D. Avis and K. Fukuda, "Reverse search for enumeration," *Discrete Appl. Math.*, vol. 65, no. 1–3, pp. 21–46, 1996.
- [51] J. R. Ullmann, "An algorithm for subgraph isomorphism," *J. ACM*, vol. 23, no. 1, pp. 31–42, 1976.
- [52] M. Houbraken, S. Demeyer, T. Michoel, P. Audenaert, D. Colle, and M. Pickavet, "The index-based subgraph matching algorithm with general symmetries (ISMAGS): Exploiting symmetry for faster subgraph enumeration," *PLoS one*, vol. 9, no. 5, 2014, Art. no. e97896.
- [53] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, "A (sub) graph isomorphism algorithm for matching large graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 10, pp. 1367–1372, Oct. 2004.
- [54] V. Carletti, P. Foggia, A. Saggese, and M. Vento, "Introducing VF3: A new algorithm for subgraph isomorphism," in *Proc. Graph-Based Representations Pattern Recognit. - 11th IAPR-TC-15 Int. Workshop*, Anacapri, Italy, 2017, vol. 10310, pp. 128–139.
- [55] W. Han, J. Lee, and J. Lee, "Turbo_(iso): Towards ultrafast and robust subgraph isomorphism search in large graph databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, New York, NY, USA, 2013, pp. 337–348.
- [56] C. McCreesh and P. Prosser, "A parallel, backjumping subgraph isomorphism algorithm using supplemental graphs," in *Proc. 21st Int. Conf. Princ. Pract. Constraint Program.*, Cork, Ireland, G. Pesant, Ed., Springer, 2015, vol. 9255, pp. 295–312.
- [57] T. Hocevar and J. Demsar, "A combinatorial approach to graphlet counting," *Bioinformatics*, vol. 30, no. 4, pp. 559–565, 2014. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btt717>
- [58] N. Kashtan, S. Itzkovitz, R. Milo, and U. Alon, "Efficient sampling algorithm for estimating subgraph concentrations and detecting network motifs," *Bioinformatics*, vol. 20, no. 11, pp. 1746–1758, 2004.
- [59] S. Wernicke, "A faster algorithm for detecting network motifs," in *Proc. 5th Int. Workshop Proc. Algorithms Bioinf.*, Springer, 2005, vol. 3692, pp. 165–177.
- [60] S. Wernicke, "Efficient detection of network motifs," *IEEE ACM Trans. Comput. Biol. Bioinf.*, vol. 3, no. 4, pp. 347–359, Oct.–Dec. 2006.
- [61] S. Wernicke and F. Rasche, "FANMOD: A tool for fast network motif detection," *Bioinformatics*, vol. 22, no. 9, pp. 1152–1153, 2006.
- [62] R. Ying, A. Z. Wang, J. You, and J. Leskovec, "Frequent subgraph mining by walking in order embedding space," in *Proc. Int. Conf. Mach. Learn. Workshops*, 2020.
- [63] R. Ying, Z. Lou, J. You, C. Wen, A. Canedo, and J. Leskovec, "Neural subgraph matching," 2020, *arXiv:2007.03092*.
- [64] N. Keriven and G. Peyré, "Universal invariant and equivariant graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 7090–7099.

- [65] O. Puny, H. Ben-Hamu, and Y. Lipman, "From graph low-rank global attention to 2-fwl approximation," 2020, *arXiv:2006.07846*.
- [66] C. Morris, G. Rattan, and M. Petra, "Weisfeiler and leman go sparse: Towards scalable higher-order graph embeddings," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21824–21840.
- [67] V. K. Garg, S. Jegelka, and T. S. Jaakkola, "Generalization and representational limits of graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3419–3430.
- [68] R. Sato, M. Yamada, and H. Kashima, "Approximation ratios of graph neural networks for combinatorial problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 4083–4092.
- [69] D. Angluin, "Local and global properties in networks of processors," in *Proc. ACM Symp. Theory Comput.*, 1980, pp. 82–93.
- [70] N. Linial, "Locality in distributed graph algorithms," *SIAM J. Comput.*, vol. 21, no. 1, pp. 193–201, 1992.
- [71] M. Naor and L. J. Stockmeyer, "What can be computed locally?," in *Proc. ACM Symp. Theory Comput.*, 1993, pp. 184–193.
- [72] P. de Haan, T. S. Cohen, and M. Welling, "Natural graph networks," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 3636–3646, 2020.
- [73] P. Li, Y. Wang, H. Wang, and J. Leskovec, "Distance encoding: Design provably more powerful neural networks for graph representation learning," *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 4465–4478, 2020.
- [74] D. Beaini, S. Passaro, V. Létourneau, W. L. Hamilton, G. Corso, and P. Liò, "Directional graph networks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 748–758.
- [75] L. Babai, P. Erdos, and S. M. Selkow, "Random graph isomorphism," *SIAM J. Comput.*, vol. 9, no. 3, pp. 628–635, 1980.
- [76] M. Fürer, "On the power of combinatorial and spectral invariants," *Linear Algebra Its Appl.*, vol. 432, no. 9, pp. 2373–2380, 2010.
- [77] M. Fürer, "On the combinatorial power of the weisfeiler-lehman algorithm," in *Proc. Int. Conf. Algorithms Complexity*, Springer, 2017, vol. 10236, pp. 260–271.
- [78] H. Dell, M. Grohe, and G. Rattan, "Lovász meets weisfeiler and leman," in *Proc. Int. Colloq. Automata, Lang., Program., Schloss Dagstuhl—Leibniz-Zentrum für Informatik*, 2018, vol. 107, pp. 40:1–40:14.
- [79] P. W. Holland and S. Leinhardt, "Local structure in social networks," *Sociol. Methodol.*, vol. 7, pp. 1–45, 1976.
- [80] N. Pržulj, "Biological network comparison using graphlet degree distribution," *Bioinformatics*, vol. 23, no. 2, pp. 177–183, 2007.
- [81] T. Milenković and N. Pržulj, "Uncovering biological network function via graphlet degree signatures," *Cancer Informat.*, vol. 6, pp. 257–273, 2008.
- [82] A. Sarajlić, N. Malod-Dognin, Ö. N. Yaveroğlu, and N. Pržulj, "Graphlet-based characterization of directed networks," *Sci. Rep.*, vol. 6, 2016, Art. no. 35098.
- [83] T. Horváth, T. Gärtner, and S. Wrobel, "Cyclic pattern kernels for predictive graph mining," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2004, pp. 158–167.
- [84] N. Shervashidze, S. Vishwanathan, T. Petri, K. Mehlhorn, and K. Borgwardt, "Efficient graphlet kernels for large graph comparison," in *Proc. Artif. Intell. Statist.*, Ser. Proceedings of Machine Learning Research, 2009, vol. 5, pp. 488–495.
- [85] F. Costa and K. De Grave, "Fast neighborhood subgraph pairwise distance kernel," in *Proc. Int. Conf. Mach. Learn.*, Omnipress, 2010, pp. 255–262.
- [86] N. M. Kriege and P. Mutzel, "Subgraph matching kernels for attributed graphs," in *Proc. Int. Conf. Mach. Learn.*, Omnipress, 2012, pp. 291–298.
- [87] H. NT and T. Maehara, "Graph homomorphism convolution," in *Proc. Int. Conf. Mach. Learn.*, Ser. Proceedings of Machine Learning Research, 2020, pp. 7306–7316.
- [88] M. R. Daredy, M. Das, and H. Yang, "motif2vec: Motif aware node representation learning for heterogeneous networks," in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 1052–1059.
- [89] R. A. Rossi, N. K. Ahmed, E. Koh, "Higher-order network representation learning," in *Companion Proc. ACM Web Conf.*, 2018, pp. 3–4.
- [90] F. Monti, K. Otness, and M. M. Bronstein, "MOTIFNET: A motif-based graph convolutional network for directed graphs," in *Proc. IEEE Data Sci. Workshop*, 2018, pp. 225–228.
- [91] A. Sankar, X. Zhang, and K. C.-C. Chang, "Meta-GNN: Meta-graph neural network for semi-supervised learning in attributed heterogeneous information networks," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2019, pp. 137–144.
- [92] J. B. Lee, R. Rossi, X. Kong, S. Kim, E. Koh, and A. Rao, "Graph convolutional networks with motif-based attention," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 499–508.
- [93] M. L. Li, M. Dong, J. Zhou, and A. M. Rush, "A hierarchy of graph neural networks based on learnable local features," 2019, *arXiv:1911.05256*.
- [94] C. Kim, H. Moon, and H. J. Hwang, "Near: Neighborhood edge aggregator for graph classification," 2019, *arXiv:1909.02746*.
- [95] D. Flam-Shepherd, T. C. Wu, P. Friederich, and A. Aspuru-Guzik, "Neural message passing on high order paths," *Mach. Learn.: Sci. Technol.*, vol. 2, no. 4, 2021, Art. no. 045009.
- [96] T. Gärtner, P. A. Flach, and S. Wrobel, "On graph kernels: Hardness results and efficient alternatives," in *Computational Learning Theory and Kernel Machines (COLT)*, vol. 2777. Berlin, Germany: Springer, 2003, pp. 129–143.
- [97] M. Neumann, R. Garnett, C. Bauckhage, and K. Kersting, "Propagation kernels: Efficient graph kernels from propagated information," *Mach. Learn.*, vol. 102, no. 2, pp. 209–245, 2016.
- [98] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels," *J. Mach. Learn. Res.*, vol. 12, pp. 2539–2561, 2011.
- [99] S. S. Du, K. Hou, R. Salakhutdinov, B. Póczos, R. Wang, and K. Xu, "Graph neural tangent kernel: Fusing graph neural networks with graph kernels," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 5724–5734.
- [100] J. Atwood and D. Towsley, "Diffusion-convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1993–2001.
- [101] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. AAAI Conf. Artif. Intell.*, AAAI Press, 2018, pp. 4438–4445.
- [102] S. Kolouri, N. Naderializadeh, G. K. Rohde, and H. Hoffmann, "Wasserstein embedding for graph learning," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [103] T. Cai, S. Luo, K. Xu, D. He, T.-Y. Liu, and L. Wang, "GraphNorm: A principled approach to accelerating graph neural network training," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1204–1215.
- [104] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [105] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2017, pp. 1024–1034.
- [106] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [107] X. Bresson and T. Laurent, "Residual gated graph convnets," 2017, *arXiv:1711.07553*.
- [108] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Velicković, "Principal neighbourhood aggregation for graph nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, 13260–13271.
- [109] M. Balcilar, P. Héroux, B. Gaüzère, P. Vasseur, S. Adam, and P. Honeine, "Breaking the limits of message passing graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 599–608.
- [110] M. Fey, J.-G. Yuen, and F. Weichert, "Hierarchical inter-message passing for learning on molecular graphs," in *Proc. Int. Conf. Mach. Learn. Workshops*, 2020.
- [111] R. Gómez-Bombarelli et al., "Automatic chemical design using a data-driven continuous representation of molecules," *ACS Central Sci.*, vol. 4, no. 2, pp. 268–276, 2018.
- [112] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar variational autoencoder," in *Proc. Int. Conf. Mach. Learn.*, Ser. Proceedings of Machine Learning Research, 2017, vol. 70, pp. 1945–1954.
- [113] W. Jin, R. Barzilay, and T. S. Jaakkola, "Junction tree variational autoencoder for molecular graph generation," in *Proc. Int. Conf. Mach. Learn.*, Ser. Proceedings of Machine Learning Research, 2018, vol. 80, pp. 2328–2337.
- [114] J. J. Irwin, T. Sterling, M. M. Mysinger, E. S. Bolstad, and R. G. Coleman, "ZINC: A free tool to discover chemistry for biology," *J. Chem. Inf. Model.*, vol. 52, no. 7, pp. 1757–1768, 2012.
- [115] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3391–3401.



Giorgos Bouritsas received the MEng Diploma in electrical and computer engineering from the National Technical University of Athens in 2017. Since 2018, he has been working toward the PhD degree in machine learning with the Department of Computing, Imperial College London, working with Prof. Michael Bronstein and Prof. Stefanos Zafeiriou. He has spent time as a visiting researcher with the Universitat Politècnica de Catalunya, Barcelona, as a research associate with the National Center for Scientific Research Demokritos, Athens, and as a visiting PhD student with the KU Leuven and École Polytechnique Fédérale de Lausanne, conducting research on a variety of topics in computer vision and machine learning. His current research interests include machine learning on graphs and general non-euclidean data. In specific, he is interested in the theoretical underpinnings of neural networks on graphs, deep probabilistic models for graph generation and compression, and their applications to network science, bioinformatics, and computer vision.



Fabrizio Frasca received the MSc degree (with distinction) in computer science and engineering from Politecnico di Milano in 2018. He is currently working toward the PhD degree with Imperial College London, under the supervision of Prof. Michael Bronstein. He has then spent time as a researcher with the Università della Svizzera Italiana, Lugano, Switzerland, visiting the Department of Computing, Imperial College London. Since 2019, he has been working for the startup company Fabula AI, acquired by Twitter. He is currently a

machine learning researcher with Twitter Cortex. His research interests include graph representation learning and network science, encompassing bioinformatics (epigenetics, drug repurposing), and computational social sciences (misinformation). His current research interests include geometric deep learning and its connections with algebraic topology, with a focus on neural approaches for the principled modeling of higher-order interactions and relational structures.



Stefanos Zafeiriou (Member, IEEE) is currently a professor of machine learning and computer vision with the Department of Computing, Imperial College London, London, U.K. Between 2016 and 2020, he was a distinguishing research fellow with the University of Oulu, under Finish Distinguishing Professor Programme. He cofounded and exited two startups, including Facesoft and Ariel AI. He has coauthored more than 80 journal papers mainly on novel machine learning methodologies applied to various domains published in the most prestigious journals in his field of research, including IEEE TPAMI and IJCV, and many papers in top conferences, including CVPR, ICCV, ECCV, and ICML. He has more than 17K+ citations to his work, h-index 60. He was an associate editor and the guest editor in more than eight journals. He was the general chair of BMVC 2017. He was the guest editor of more than 10 journal special issues and co-organised more than twenty workshops or special sessions on specialised computer vision and machine learning topics in top venues, including CVPR or FG or ICCV or ECCV or NeurIPS. He is currently the area chair in the top venues of his field. His students are frequent recipients of very prestigious and highly competitive fellowships, such as the Google Fellowship, Intel Fellowship, and Qualcomm Fellowship. He was the recipient of the Prestigious Junior Research Fellowships from Imperial College London in 2011, President's Medal for Excellence in Research Supervision for 2016, Google Faculty Research Awards, and Amazon AWS ML Research Award. He is an EPSRC Early Career research fellow.



Michael Bronstein (Fellow, IEEE) received the PhD degree from Technion in 2007. He is currently the deepmind professor of AI with the University of Oxford and the head of Graph Learning Research with Twitter. He was previously a professor with Imperial College London and has held visiting appointments with Stanford, MIT, and Harvard, and three Institutes for Advanced Study, with TUM as a rudolf diesel fellow (2017–2019), with Harvard as a radcliffe fellow (2017–2018), and with Princeton as a short-time scholar (2020). He was the recipient of the Royal Society Wolfson Research Merit Award, Royal Academy of Engineering Silver Medal, five ERC grants, two Google Faculty Research awards, and two Amazon AWS ML Research awards. He is a member of Academia Europaea, a fellow of IAPR, BCS, and ELLIS, ACM distinguished speaker, and World Economic Forum young scientist. In addition to his academic career, he is a serial entrepreneur and founder of multiple startup companies, including Novafora, Invision (acquired by Intel, 2012), Videocites, and Fabula AI (acquired by Twitter, 2019). He was a principal engineer with Intel Perceptual Computing and was one of the key developers of Intel RealSense technology.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.